Compresión de imágenes con Quadtree mediante Divide y Vencerás

Diseño y Análisis de Algoritmos – Examen práctico

Integrantes: _____

Octubre 2026

Repositorio: código, experimentos (`results/results.csv`) y evidencia (`evidence/`).

A. Modelado del problema

Entrada. Imagen digital de $H \times W$ píxeles con C canales (C=3 RGB o C=1 grises), matriz `uint8` de forma (H, W, C) y una tolerancia $\tau \geq 0$. El tamaño de entrada es **N = H·W** píxeles (con C constante); se usa también $L = \max(H,W)$. Las dimensiones son arbitrarias (no necesariamente potencias de 2). **Salida.** (i) Un Quadtree cuyas hojas particionan la imagen en rectángulos con un color representativo, (ii) un archivo binario con esa estructura y (iii) la imagen reconstruida $\hat{I}$ de dimensiones originales. **Restricciones.** Cada píxel pertenece a exactamente una hoja; no hay regiones vacías; el error de reconstrucción debe poder cuantificarse y estar acotado por τ.

Representación y similitud de colores. Cada píxel es un vector $p \in \{0..255\}^C$. El color representativo de una región R de n píxeles es la media por canal redondeada: $m_c = \lfloor (2 \cdot S1_c + n) / (2n) \rfloor$, con $S1_c = \Sigma_i\, p_{i,c}$. La disimilitud de la región es el error cuadrático medio contra ese color: $\mathrm{MSE}(R) = (1/(nC)) \Sigma_c \Sigma_i (p_{i,c} - m_c)^2$. **Criterio de uniformidad:** R es uniforme $\Leftrightarrow \mathrm{MSE}(R) \leq \tau$. Se justifica porque (a) es la misma métrica con que se mide la calidad final, (b) $\tau = 0$ equivale a exigir que todos los píxeles sean idénticos (compresión sin pérdida), (c) usa todos los píxeles (no solo los extremos) y (d) se calcula con aritmética entera exacta a partir de Σp y Σp^2, lo que permite el cálculo en O(1) con imágenes integrales.

B. Diseño algorítmico

Divide y Vencerás. `build(R)`: **Dividir** – evaluar el criterio sobre R; si no se cumple, partir R en cuadrantes. **Resolver** – llamar recursivamente a `build` en cada cuadrante; **casos base**: región uniforme (hoja con su color) o región de 1 píxel (siempre uniforme, MSE = 0). **Combinar** – el nodo padre agrupa las referencias a sus hijos y agrega contadores (nodos, hojas, profundidad). Nodo: `(x, y, w, h, color | None, hijos | None)`; las hojas guardan el color y los nodos internos entre 2 y 4 hijos (orden NO, NE, SO, SE).

Dimensiones no potencia de dos. Una región $w \times h$ se parte en $w_1 = \lceil w/2 \rceil$, $w_2 = w - w_1$ columnas y $h_1 = \lceil h/2 \rceil$, $h_2 = h - h_1$ filas. Los rectángulos $w_1 \times h_1$, $w_2 \times h_1$, $w_1 \times h_2$, $w_2 \times h_2$ son disjuntos y su unión es R (sin superposición ni omisión). Si w=1 (o h=1) el cuadrante con w_2=0 (h_2=0) se omite, por lo que nunca se crean nodos vacíos; esa regla también da árboles de 2 hijos en tiras de un píxel. Para $w \geq 2$ se cumple $w_1 < w$ (análogo en h), así que cada hijo tiene estrictamente menos píxeles que su padre: la recursión progresa hacia el caso base.

Reconstrucción. Recorrido en preorden del árbol y relleno de cada hoja con su color: $\Theta(\#\text{nodos} + N \cdot C)$. **Formato de almacenamiento (bits reales).** Cabecera de 40 bits (ancho 16, alto 16, canales 8); luego el árbol en preorden: 1 bit de bandera por nodo de área > 1 (0 = hoja, 1 = interno) y $8 \cdot C$ bits por hoja; relleno a byte. No se guardan coordenadas ni punteros: el decodificador repite la misma partición. Tamaño: $B = \lceil (40 + F + 8 \cdot C \cdot L_h)/8 \rceil$ bytes con F = #nodos de área > 1 y L_h = #hojas. **Razón de compresión** = $H \cdot W \cdot C \,/\, B$ (> 1 comprime, < 1 expande); también se reportan bits por píxel y MSE/PSNR ($\mathrm{PSNR} = 10 \log_{10}(255^2/\mathrm{MSE})$). Se implementaron dos variantes del criterio con idéntico árbol resultante: **naive** (recorre los píxeles de la región) e **integral** (tablas de sumas acumuladas de p y p^2).

C. Correctitud y complejidad

Correctitud. *Lema 1 (partición):* los cuatro cuadrantes son disjuntos, cubren R y tienen área estrictamente menor (se verifica arriba y con una prueba exhaustiva para $1 \leq w,h < 20$). *Lema 2 (terminación):* el área es un entero positivo que decrece en cada llamada; con área 1 hay caso base. *Proposición:* `build(R)` devuelve un subárbol cuyas hojas particionan R y cuyo color cumple $\mathrm{MSE} \leq \tau$. Por inducción sobre el área: en el caso base es inmediato (una hoja; con 1 píxel el MSE es $0 \leq \tau$); en el caso inductivo, por el Lema 1 los subproblemas particionan R y por hipótesis sus hojas particionan cada cuadrante y cumplen el criterio. *Corolario (cota de error):* $\mathrm{MSE}_{\text{global}} = \Sigma_{\text{hojas}}\, \mathrm{SSE} \,/\, (NC) \leq \Sigma\, \tau\, n_i C \,/\, (NC) = \tau$, porque la comparación usa el color *almacenado* (ya redondeado); con $\tau = 0$ todas las hojas son constantes y la reconstrucción es exacta. *Reconstrucción:* cada hoja pinta su rectángulo y las hojas particionan la imagen, luego cada píxel se escribe una vez. Verificación empírica: 264 corridas, $\mathrm{MSE} \leq \tau$ en todas: **sí**; τ=0 sin pérdida: **sí**; decodificar(codificar(árbol)) idéntico al original en todas.

Operación fundamental y costo del chequeo de uniformidad. Una *consulta* de uniformidad sobre una región de n píxeles cuesta $\Theta(n)$ lecturas de píxel en el modo naive (hay que ver todos, no existe corte temprano porque el criterio depende de la media) y $\Theta(1)$ lecturas de tabla en el modo integral, tras $\Theta(N)$ de preprocesamiento. Un modelo basado solo en #llamadas ignoraría ese costo.

Recurrencias (imagen cuadrada, n = píxeles de la región, peor caso: toda región no uniforme hasta 1 píxel):

`naive: T(n) = 4·T(n/4) + θ(n), T(1) = θ(1) ⇒ a = b = 4, f(n) = θ(n^{log₄4}) ⇒ caso 2 del método maestro ⇒ T(N) = θ(N log N) = θ(N·log L)`

`integral: T(n) = 4·T(n/4) + θ(1), T(1) = θ(1) ⇒ f(n) = O(n^{1−ε}) ⇒ caso 1 ⇒ T = θ(N) (+ θ(N) de tablas) = θ(N)`

Árbol de recursión: el nivel k tiene $\leq 4^k$ nodos de $N/4^k$ píxeles; naive paga $\Theta(N)$ por nivel y hay $\log_4 N + 1 = \log_2 L + 1$ niveles; integral paga $\Theta(1)$ por nodo y $\Sigma 4^k = (4N-1)/3$. **Cota general** (cualquier dimensión): los nodos de una misma profundidad son disjuntos, así que naive lee $\leq N$ píxeles por nivel y la profundidad es $\leq \lceil \log_2 L \rceil$, de modo que $T \leq N(\lceil \log_2 L \rceil + 1) = O(N \log L)$; además $\#\text{nodos} \leq 2N-1$ (cada nodo interno tiene ≥ 2 hijos), luego integral es $O(N)$. **Mejor caso:** imagen uniforme: naive = $\Theta(N)$ (un solo barrido, 1 nodo), integral = $\Theta(N)$ por el preprocesamiento y $\Theta(1)$ de recursión. **Peor caso:** ruido con $\tau = 0$ (todas las regiones se subdividen): naive $\Theta(N \log N)$ con exactamente $N(\log_2 L + 1)$ lecturas; integral $\Theta(N)$, con $(4N-1)/3$ nodos. **Espacio.** Árbol: $\Theta(\#\text{nodos}) \leq O(N)$ en el peor caso y O(1) en el mejor; pila de recursión: $\Theta(\text{profundidad}) = O(\log L)$; integral añade $2(H+1)(W+1)C$ enteros de 64 bits ($\Theta(N)$, un costo de memoria que sí es relevante). **Costos distintos:** *construir* como arriba; *recorrer* el árbol $\Theta(\#\text{nodos})$ ($\Omega(1)$, $O(N)$); *reconstruir* $\Theta(\#\text{nodos} + N) = \Theta(N)$ siempre, pues hay que escribir $N \cdot C$ bytes; *codificar/decodificar* $\Theta(\#\text{nodos})$.

D. Resultados experimentales

Diseño. Imágenes sintéticas reproducibles (semilla fija): *uniform* (un color), *blocks* (regiones de colores definidos: rectángulos, círculo, banda inclinada), *detail* (degradados, textura, formas y ruido gaussiano σ=4) y *noise* (ruido uniforme, peor caso). Tamaños 64, 128, 256 (más 32 y 512 y tres tamaños no potencia de dos: 97×61, 200×150, 255×255, ancho×alto), tolerancias $\tau \in \{0, 5, 25, 100, 400, 1600\}$ y ambos modos de uniformidad: **264** corridas en `results/results.csv` (tiempo = mínimo de 3 repeticiones, 2 en 512^2; Python 3 + NumPy; las cifras de tiempo son específicas de esta máquina y de esta implementación).

Tabla 1. *Imágenes de 256×256 (196 608 bytes sin comprimir). Las 4 imágenes × 6 tolerancias; razón < 1 = el Quadtree* **expande** *la imagen.*

contenido	τ	nodos	hojas	prof.	bytes	razón	MSE	PSNR dB	t naive ms	t integ. ms
uniform	0	1	1	0	9	21845.3	0.0	∞	1.8	5.2
	5	1	1	0	9	21845.3	0.0	∞	1.8	5.0
	25	1	1	0	9	21845.3	0.0	∞	1.8	4.9
	100	1	1	0	9	21845.3	0.0	∞	1.8	5.0
	400	1	1	0	9	21845.3	0.0	∞	1.8	5.2
	1600	1	1	0	9	21845.3	0.0	∞	1.8	4.9
blocks	0	4785	3589	8	11092	17.7	0.0	∞	56.5	53.7
	5	4785	3589	8	11092	17.7	0.0	∞	58.0	52.6
	25	4785	3589	8	11092	17.7	0.0	∞	56.4	50.9
	100	4745	3559	8	10998	17.9	0.7	49.7	53.4	53.7
	400	3833	2875	8	8888	22.1	35.0	32.7	47.4	43.1
	1600	1561	1171	8	3627	54.2	323.8	23.0	21.8	22.0
detail	0	87381	65536	8	199344	0.99	0.0	∞	973	965
	5	83409	62557	8	190407	1.03	0.2	54.5	916	915
	25	21021	15766	8	49010	4.01	13.7	36.8	246	233
	100	4577	3433	8	10634	18.5	53.4	30.9	57.1	53.8
	400	3925	2944	8	9094	21.6	124.8	27.2	50.0	45.1
	1600	2737	2053	8	6338	31.0	401.5	22.1	35.4	32.0
noise	0	87381	65536	8	199344	0.99	0.0	∞	986	942
	5	87381	65536	8	199344	0.99	0.0	∞	953	998
	25	87381	65536	8	199344	0.99	0.0	∞	1055	1048
	100	87381	65536	8	199344	0.99	0.0	∞	1110	1035
	400	87377	65533	8	199335	0.99	0.0	67.9	1095	1119
	1600	85569	64177	8	195267	1.01	34.5	32.8	1053	1013

Tabla 2. *Crecimiento del costo con el tamaño (τ = 0 y τ = 25). "lecturas naive/N" = lecturas de píxel del criterio entre N (≈ $\log_2 L+1$ en el peor caso).*

contenido	tam.	N	nodos (τ=0)	lecturas naive/N	t naive ms	t integral ms	nodos (τ=25)	t naive ms	t integral ms
uniform	32²	1024	1	1.00	0.0	0.0	1	0.1	0.1
	64²	4096	1	1.00	0.1	0.1	1	0.1	0.2
	128²	16384	1	1.00	0.3	0.8	1	0.3	1.0
	256²	65536	1	1.00	1.8	5.2	1	1.8	4.9
	512²	262144	1	1.00	7.3	35.2	1	9.6	36.6
detail	32²	1024	1365	6.00	15.5	15.5	1125	13.1	12.7
	64²	4096	5461	7.00	58.3	56.0	3981	42.7	39.4
	128²	16384	21845	8.00	249	293	10453	144	127
	256²	65536	87381	9.00	973	965	21021	246	233
	512²	262144	349525	10.00	4534	4377	30701	467	421
noise	32²	1024	1365	6.00	16.1	14.9	1365	15.3	15.0
	64²	4096	5461	7.00	50.2	47.8	5461	50.1	49.6
	128²	16384	21845	8.00	297	292	21845	263	220
	256²	65536	87381	9.00	986	942	87381	1055	1048
	512²	262144	349525	10.00	4543	5530	349525	4491	3825

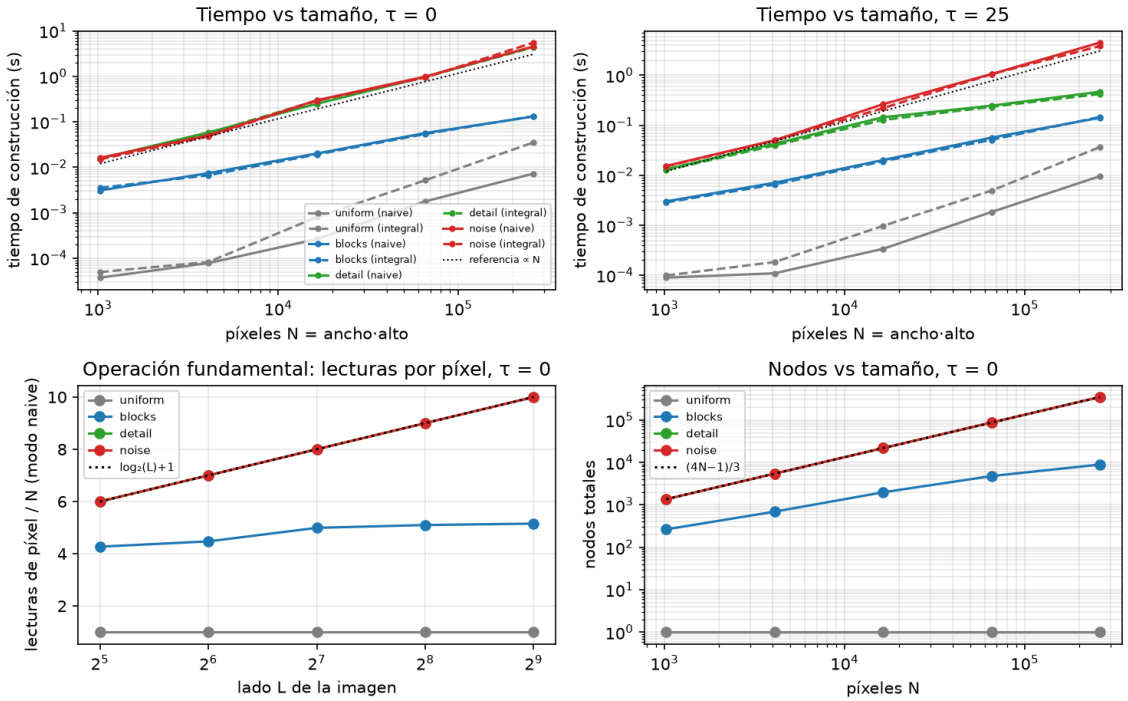


Figura 1. *Arriba: tiempo de construcción vs N (escala log-log; sólido = naive, discontinuo = integral). Abajo-izq.: lecturas de píxel por píxel (modo naive, τ=0) vs $\log_2 L+1$. Abajo-der.: nodos vs N frente a (4N−1)/3.*

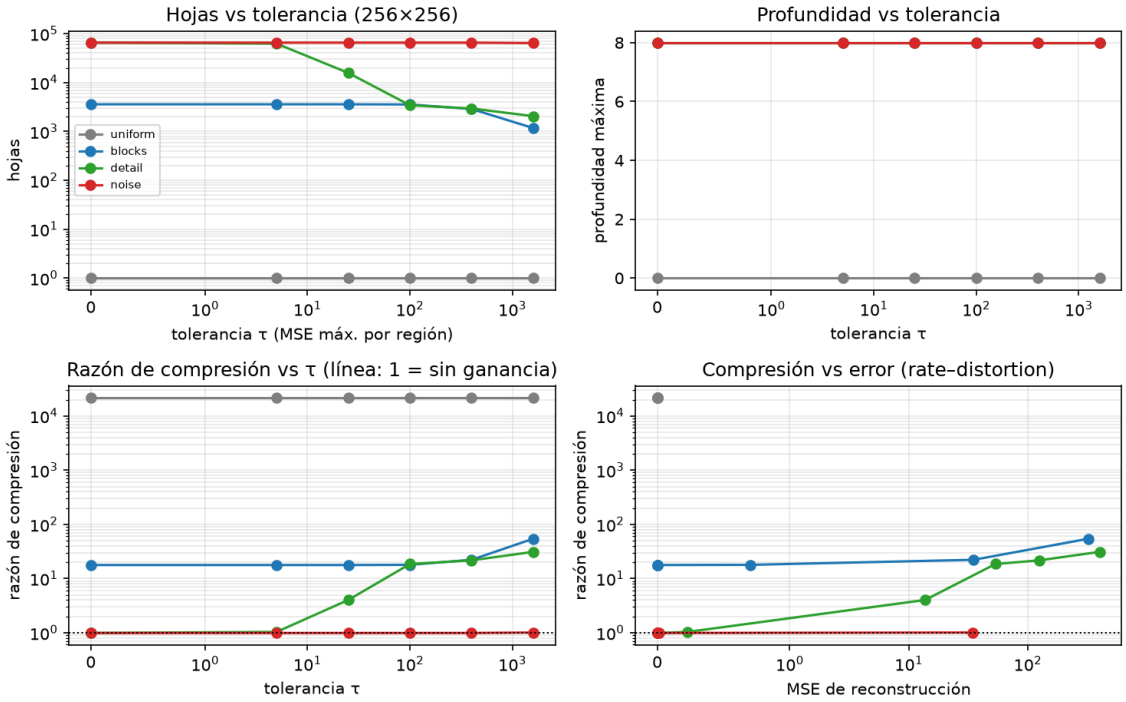
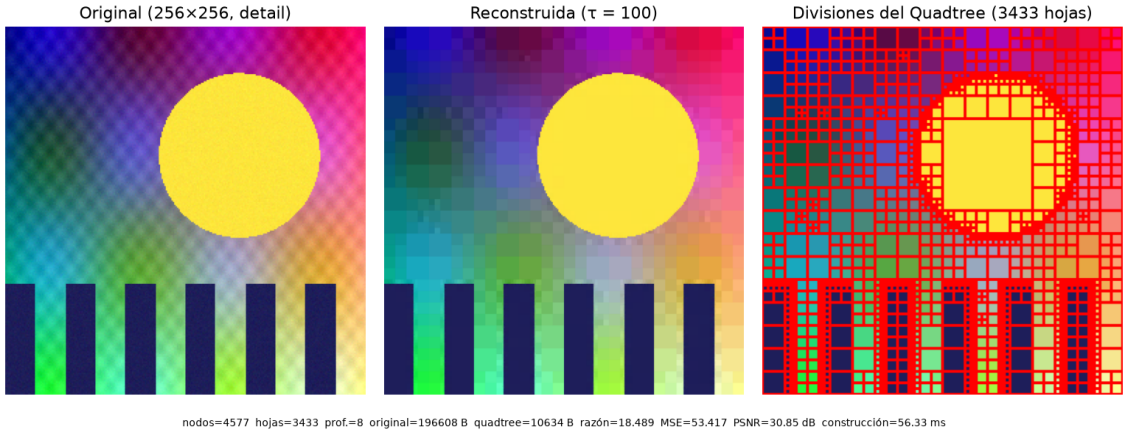


Figura 2. *Imágenes de 256×256: hojas, profundidad y razón de compresión vs tolerancia, y curva compresión–error.*

Evidencia de funcionamiento (carpeta `evidence/`): original, reconstruida, divisiones y métricas para cinco configuraciones (ev1 blocks 256² τ=0; ev2 detail 256² τ=100; ev3 detail 256² τ=400; ev4 noise 128² τ=0, donde no se comprime; ev5 detail 150×200 τ=100, dimensiones no potencia de 2); `gallery_detail_256.png` muestra el barrido de tolerancias.



Interpretación.

(1) Tamaño de la imagen y costo. Con $\tau = 0$ las imágenes *detail* y *noise* producen el árbol completo, con $(4N-1)/3$ nodos (5461 → 21845 → 87381 → 349525 para L = 64…512): el tiempo se multiplica por ≈ 4.5 al duplicar L (×4 sería lineal en N) y cuesta ≈ 11–16 µs por nodo, es decir, en la práctica el tiempo es proporcional al *número de nodos*. En *blocks* los nodos crecen solo ×2–2.7 por duplicación de L (693 → 1981 → 4785 → 8937): son proporcionales a la longitud de las fronteras entre regiones ($\Theta(L)$), no al área. En *uniform* el árbol es un solo nodo para cualquier tamaño y el único costo es leer los N píxeles. El contador de operaciones confirma el análisis: el modo naive hace exactamente 10.00 = $\log_2 512+1$ lecturas por píxel en ruido (Fig. 1, abajo-izq.) y 4.5–5.2 en *blocks*, es decir $\Theta(N \log L)$ en el peor caso; el modo integral lee cada píxel una sola vez ($\Theta(N)$).

Naive frente a integral. Aunque las lecturas difieren hasta ×10, los tiempos medidos difieren poco (512²/ruido, τ=0: 4.54 s vs 5.53 s): en esta implementación en Python el costo por nodo (≈ decenas de µs de intérprete) domina sobre el barrido vectorizado con NumPy (≈ 28 ns por píxel, medido en *uniform* 512²). La ventaja asintótica del modo integral solo se aprecia en los contadores y se vuelve visible si la región inspeccionada es grande y el árbol profundo; en el mejor caso ocurre lo opuesto: en *uniform* 512² naive tarda 7.3 ms e integral 35.2 ms porque este último construye las tablas ($\Theta(N)$ de tiempo y memoria) aunque no las necesite.

(2) Efecto de la tolerancia (256×256). En *detail* las hojas bajan de 65536 (τ=0) a 15766 (25), 3433 (100), 2944 (400) y 2053 (1600); la mayor caída ocurre cuando τ supera la varianza del ruido ($\sigma^2 = 16$) y del degradado local, y luego hay rendimientos decrecientes. En *blocks* el número de hojas (3589) no cambia hasta $\tau = 25$ porque las regiones son constantes: solo las hojas de frontera cambian cuando τ deja de exigir uniformidad. En *noise* la varianza por canal es ≈ 5461 ≫ 1600, por lo que casi ninguna región es uniforme (64177 hojas con τ=1600): la tolerancia no ayuda. La profundidad máxima se mantiene en $\log_2 256 = 8$ salvo en *uniform* (0). El MSE real alcanzado en *detail* es solo ≈ 36 % de τ en promedio: τ es una cota por región, no el error esperado.

(3) Compresión frente a calidad (detail 256²). τ=25: razón 4.01, PSNR 36.8 dB; τ=100: 18.5, 30.9 dB; τ=400: 21.6, 27.2 dB; τ=1600: 31.0, 22.1 dB. La curva compresión–error (Fig. 2, abajo-der.) es muy empinada al inicio y se aplana: pasar de 100 a 1600 solo multiplica la razón por 1.7 a costa de ≈ 9 dB. Con $\tau = 0$ la reconstrucción es exacta (MSE = 0) pero solo hay ganancia si hay regiones constantes (*blocks*: 17.7×).

(4) Casos en que no se reduce el almacenamiento. En 38 de 132 combinaciones imagen × tolerancia el archivo es mayor o igual que la imagen original (contenidos: detail, noise). Con ruido y $\tau = 0$ el Quadtree ocupa 1.4 % *más* que la imagen cruda: hay una hoja de 24 bits por píxel más ≈ N/3 banderas y la cabecera. Frente a PNG (referencia opcional, columna `png_bytes`), el Quadtree sin pérdida es peor en *blocks* (11092 B contra 1017 B), porque cada frontera diagonal o curva se descompone en una "escalera" de hojas; en *detail* con τ=100 ocupa 10634 B contra 113020 B de PNG, pero con pérdida (PSNR 30.9 dB), así que no es una comparación equivalente.

(5) ¿Cuándo conviene un Quadtree? Cuando la imagen tiene regiones grandes casi constantes con fronteras alineadas o poco numerosas (gráficos, mapas, diagramas, máscaras, texturas planas) y se acepta un error acotado; el costo de construcción crece con el número de nodos, y el árbol resultante además ofrece acceso jerárquico a la distribución espacial. No conviene con ruido o texturas finas ni con $\tau = 0$ sobre contenido de alta frecuencia, donde el costo aumenta a $\Theta(N \log N)$ (naive) o $\Theta(N)$ con mayor memoria (integral) y el archivo no se reduce.

E. Conclusiones

• **Ventajas.** El Divide y Vencerás da una solución simple, demostrablemente correcta y con cota de error explícita (MSE ≤ τ); es adaptativo (más hojas donde hay detalle); el árbol es una representación jerárquica reutilizable y la reconstrucción cuesta $\Theta(N)$. Con imágenes uniformes o por regiones la compresión es muy alta (de 17× sin pérdida hasta > 20 000× en una imagen uniforme).

• **Limitaciones.** El rendimiento depende del contenido, no solo del tamaño: el peor caso (ruido) tiene $(4N-1)/3$ nodos y puede *expandir* el archivo; la partición rígida en cuadrantes no sigue bordes oblicuos o curvos; la elección de τ no garantiza un nivel de calidad percibido (MSE ≠ calidad visual); el modo integral cambia tiempo por $\Theta(N)$ de memoria. Los tiempos absolutos son de un prototipo en Python y no son comparables con un compresor en C.

• **Recomendación.** Usar integral cuando los árboles sean profundos y las regiones grandes; naive si la imagen suele ser casi uniforme o la memoria es limitada. Elegir τ en el "codo" de la curva compresión–error (aquí $\tau \approx$ 25–100 en *detail*) y no usar el Quadtree sin pérdida sobre contenido ruidoso. Trabajo futuro: podar o fusionar hojas por criterio de tasa–distorsión, codificar los colores con predicción o entropía, y usar particiones adaptativas (árboles binarios) que sigan mejor las fronteras.

Referencias

[1] Finkel, R. A. y Bentley, J. L. (1974). Quad trees: a data structure for retrieval on composite keys. *Acta Informatica*, 4(1), 1–9.

[2] Samet, H. (1984). The quadtree and related hierarchical data structures. *ACM Computing Surveys*, 16(2), 187–260.

[3] Crow, F. C. (1984). Summed-area tables for texture mapping. *Computer Graphics (SIGGRAPH '84)*, 18(3), 207–212.

[4] Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). *Introduction to Algorithms* (4.ª ed.). MIT Press. (Método maestro, cap. 4.)

[5] Gonzalez, R. C. y Woods, R. E. (2018). *Digital Image Processing* (4.ª ed.). Pearson. (Razón de compresión, MSE, PSNR.)